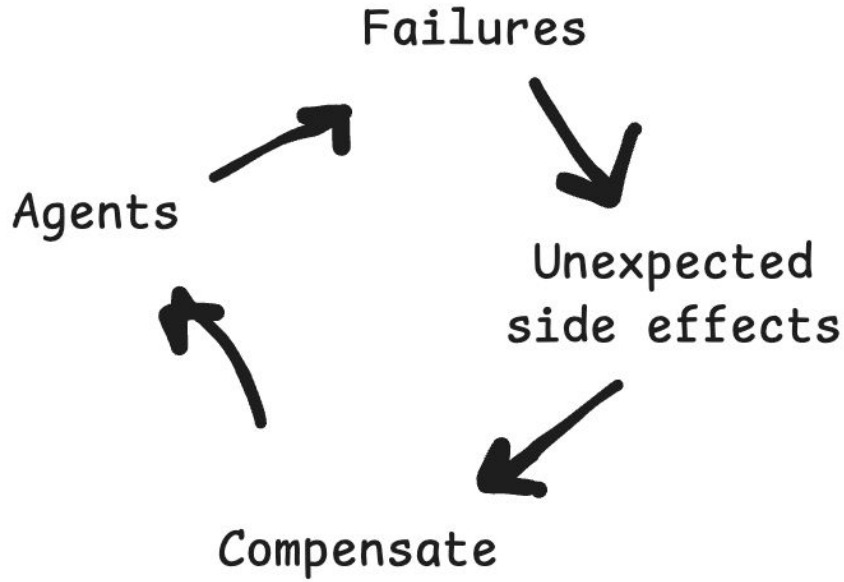


Robust Agent Compensation (RAC): Teaching AI Agents to Compensate

Srinath Perera

*WSO2(VP, Research),
Apache Software Foundation (Member)*



Outcomes depend on

- the problem
- the nature of disruptions
- the prompt
- the model

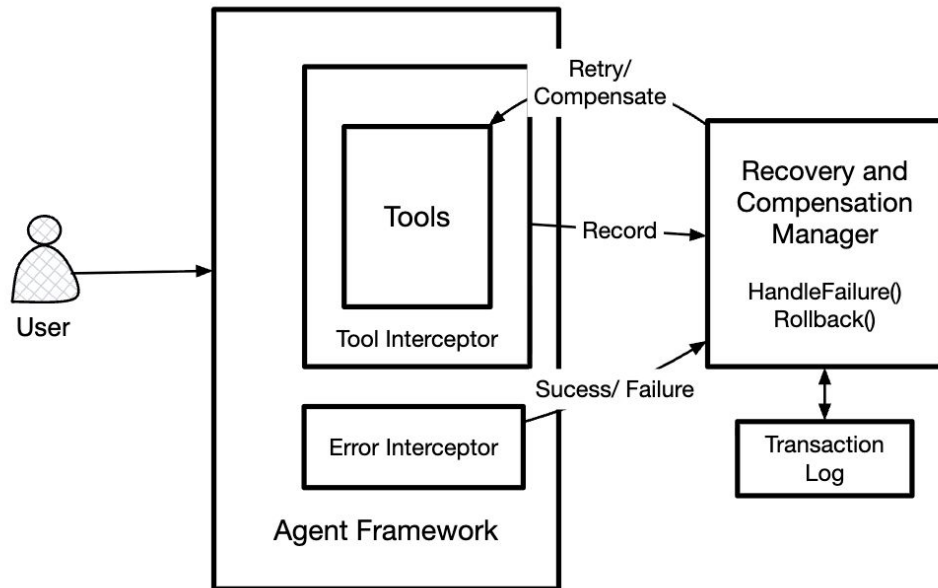
Travel booking as an example!! → flights successful,
but hotel failed

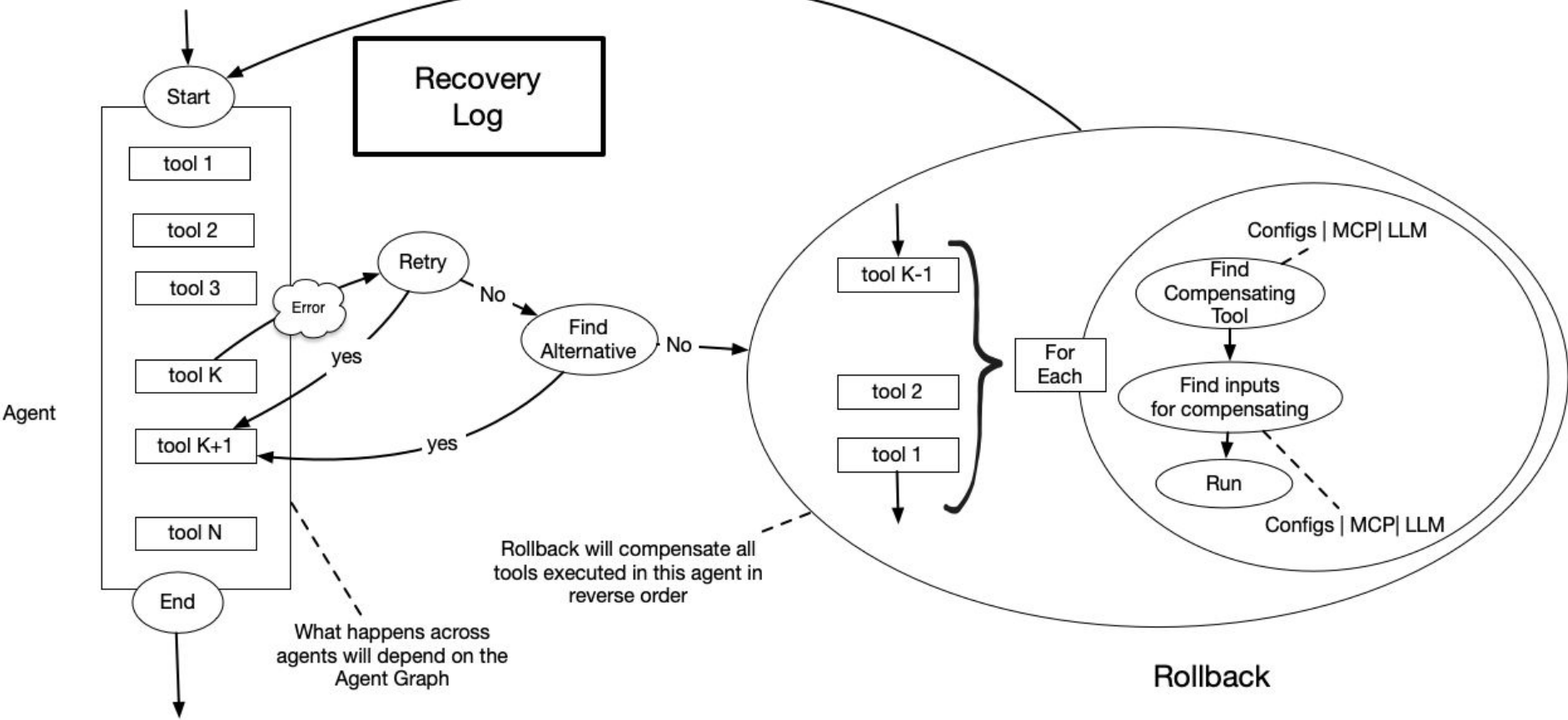
Existing Solutions

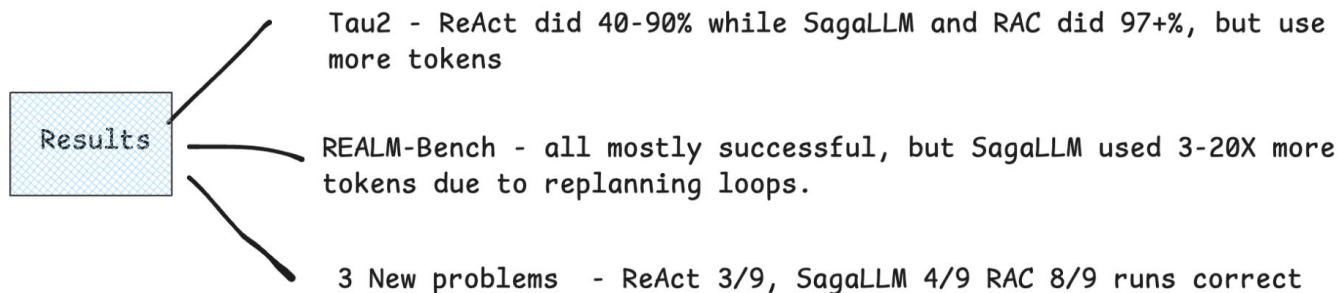
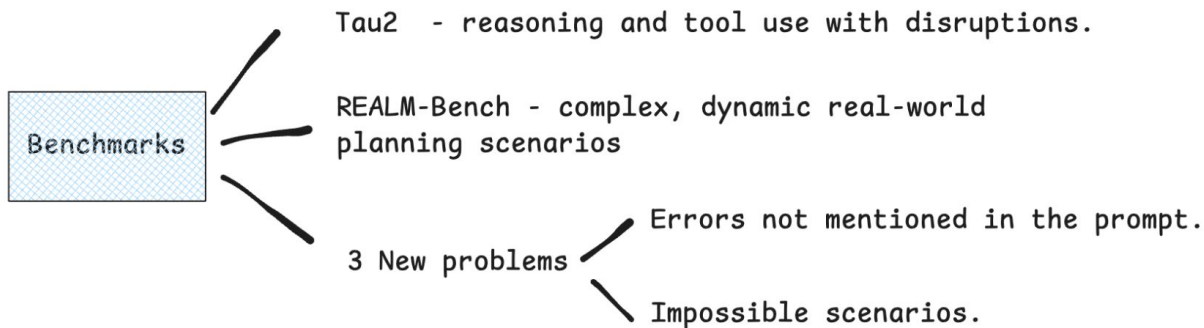
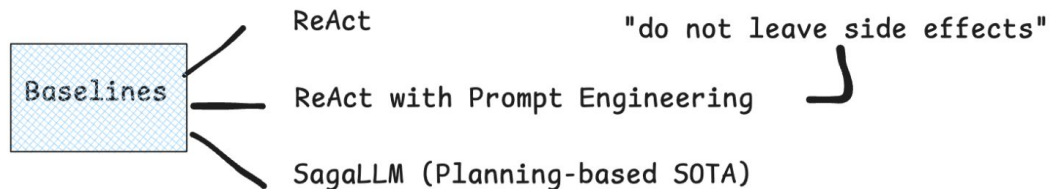
- Planning (e.g., SagaLLM)
 - Can lead to failures and costly replanning loops (as we will see in the evaluation)
- Developer has to handle
 - Because ReAct agent execution is dynamic, handling this with manual code is non-trivial
- LLMs may improve the point that errors are rare

Proposed Solution

- Ideas
 - Adept the log based recovery + compensation to Agent
 - Let React planning work, while tracking what happens based on the recovery log, and step in when needed
- It can work with the existing LangChain, LanGraph code etc., with no changes.
- Design
 - Keeps a recovery log and uses that to compensate

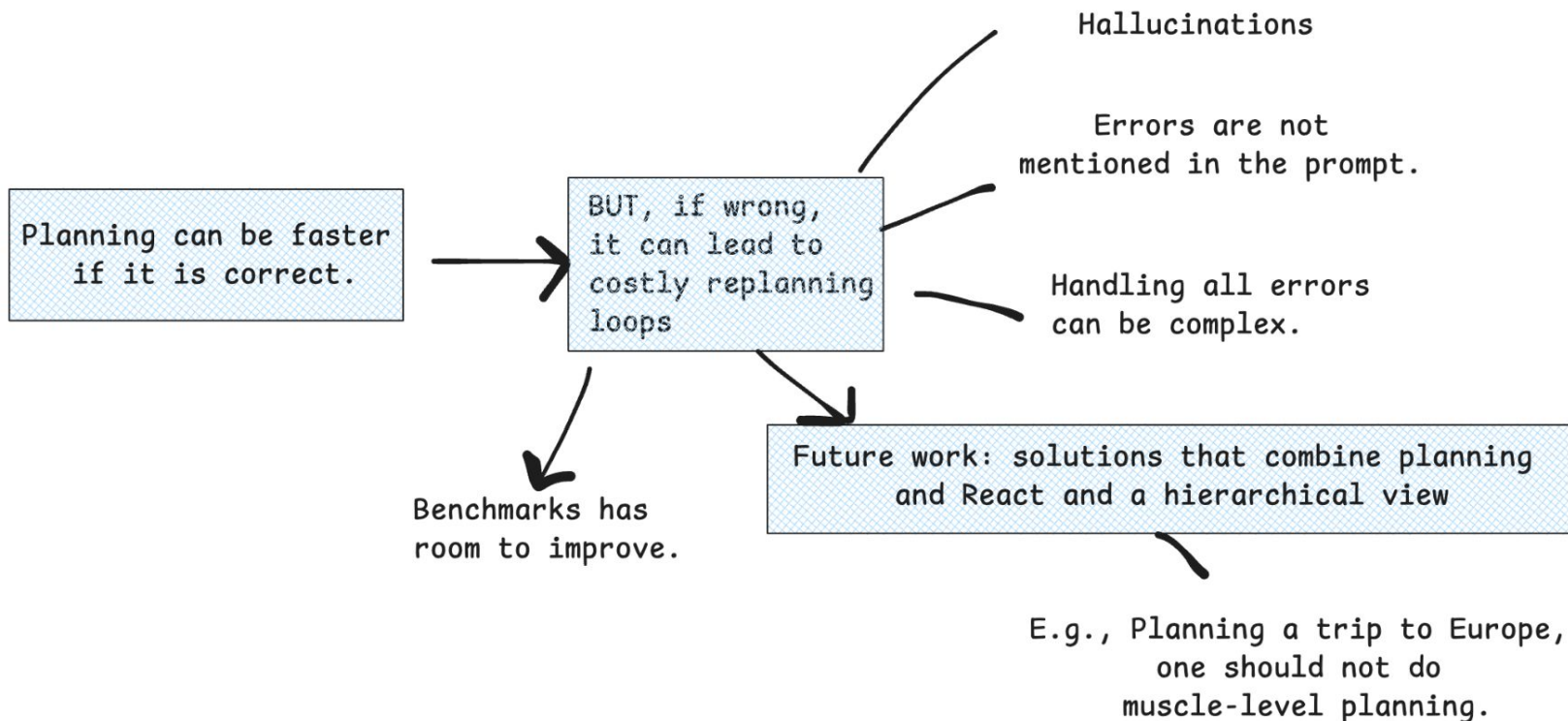






Observations and Lessons Learned

Understanding "Pre-planning vs. ReAct" tradeoffs



Observations and Lessons Learned

Clear view of history due to the recovery log

Why RAC + ReAct
outperforms vanilla ReAct?

Do not have to handle failures.

Recommended Config: MCP for finding a competing tool
+ LLM for input finding to the competing tool
(only one-time work at MCP configuration time)

RAC

guarantees on unexpected side effects

With minimal additional work

No or small code changes for most agentic platforms

Who
We are?

WSO2 Research is a industry lab working on (AI + System) + (AI + Software Engineering)

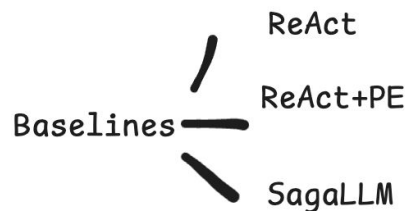
WSO2 is a company building open source AI and cloud platforms with customers in 93+ countries and 100m+ ARR

BTW - we have <https://wso2.com/agent-platform/>, a open platform for the full lifecycle of enterprise-grade AI agents

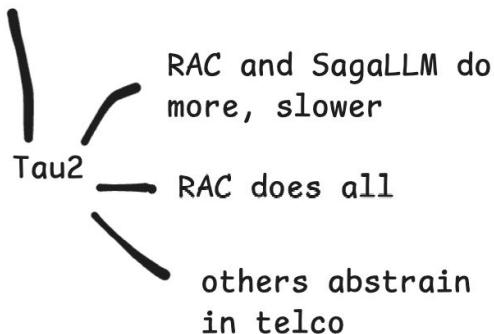
Additional Slides

Evaluation with Tau2 Bench

ig, or use the nano tool



Let you abstain



Category	Framework	Success %	Compl. %	Time (s)	Tokens
Airline	LG	82	-	52	48k
	LG(PE)	82	-	69	62k
	SagaLLM	97	-	76	80k
	RAC	97	81*	50	77k
Retail	LG	90	-	28	49k
	LG(PE)	91	-	30	52k
	SagaLLM	98	-	65	71k
	RAC	100	89*	22	66k
Telecom	LG	45	1	16	49k
	SagaLLM	100	2	32	100k
	RAC	99	99	58	176k

Table 1: Performance Metrics for TAU Benchmark Tasks(results from 3 Iterations)

Evaluation with RELAM-Bench

- Disruptions are mentioned in the prompt
- RAC/ReAct matches in token/ time, while SagaLLM take 3-20X more tokens
- With REALAM-bench (static - means errors are known), RAC matches ReAct in tokens and time due to replanning
- Replanning happens due to hallucinations or planning not capturing errors.

ID	Framework	Result	Tokens	Time (s)	Comp.
P5	LG	3/3	11k	17	—
	LG(PE)	3/3	28k	24	—
	SagaLLM	3/3	250k	646	—
	RAC	3/3	10k	15	—
	RAC_M	3/3	14k	18	—
P6	LG	3/3	6k	17	—
	LG(PE)	3/3	6k	14	—
	SagaLLM	3/3	238k	579	137
	RAC	3/3	9k	15	—
	RAC_M	3/3	9k	16	—
P8	LG	3/3	6k	17	—
	LG(PE)	3/3	13k	18	—
	SagaLLM	3/3	56k	463	16
	RAC	3/3	33k	32	8
	RAC_M	3/3	31k	34	6.33
P9	LG	3/3	24k	28	—
	LG(PE)	3/3	74k	44	—
	SagaLLM	3/3	52k	106	33
	RAC	3/3	9k	14	0
	RAC_M	3/3	29	30	6.00
P11	LG	3/3	3k	14	—
	LG(PE)	3/3	6k	15	—
	SagaLLM	2/3	67k	302	0
	RAC	3/3	12k	20	0
	RAC_M	3/3	38k	54	3.33

Table 2: REALM-Bench Tasks (results from 3 Iterations)

Results with New Problems

- New problems
 - Problems that have dynamic failures (not mentioned in the problem statement)
 - Impossible problems
- Observations
 - LG(ReAct) fails
 - Some runs fail on all, RAC does better

ID	Framework	Result	Tokens	Time (s)	Comp.
P12	LG	0/3	1k	11	—
	LG(PE)	3/3	17k	59	—
	SagaLLM	1/3	71k	532	1
	RAC_M	3/3	10k	21	—
	RAC	3/3	9k	18	—
P13	LG	0/3	0k	3	—
	LG(PE)	0/3	66k	43	—
	SagaLLM	2/3	80k	202	0
	RAC_M	3/3	88k	70	0.33
	RAC	3/3	76k	58	0
P14	LG	0/3	—	18	0
	LG(PE)	0/3	—	17	0
	SagaLLM	1/3	95k	173	2
	RAC	2/3	189k	74	15

Table 3: Tasks with Dynamic Failures (results from 3 Iterations)

Contributions

- We propose RAC, a log-based recovery paradigm (providing a safety net) implemented through an architectural extension that can be applied to most Agent frameworks with minimal changes to existing code to support reliable executions (avoiding unintended side effects).
- Identify and extend benchmarks for testing the agent for unintended execution side effects when faced with disruptions that are not mentioned in the problem description.
- Propose an extension to the Model Context Protocol (MCP) to describe compensation pairs, creating an interoperable way for agent frameworks to discover those pairs.
- We present evidence that decoupling recovery from the ReAct Agents can enable them to solve harder problems while increasing robustness, uncovering strong directions for the design of a robust agent execution framework.
- We provide an open-source reference implementation of RAC (<https://github.com/wso2-incubator/research-rac>).